

Hour Booking System - Plain Java (Servlets + Raw JDBC) Guide

Hour Booking System - Plain Java Version (No Spring)

Same app as the Spring Boot version, rebuilt with **plain Java**: Jakarta Servlets instead of Spring MVC, and raw JDBC (Connection / PreparedStatement) instead of Spring Data JPA/Hibernate. Same MySQL schema, same HTML/CSS/JS frontend.

Step 0: Create the Project

1. Create a **Maven Web Application** project (not Spring Initializr this time):
 - In your IDE: New Project -> Maven -> maven-archetype-webapp, or just create the folder structure below by hand.
 - **Group**: com.example
 - **Artifact**: hourbooking-plain
 - **Packaging**: war
2. Folder layout:

hourbooking-plain/
 pom.xml
 src/main/java/com/example/hourbooking/
 db/DBConnection.java
 dao/*.java
 servlet/*.java
 src/main/webapp/
 WEB-INF/web.xml
 index.html, register.html, staff-dashboard.html, admin-dashboard.html
 style.css, script.js
3. Install a servlet container: **Apache Tomcat 10.x** (needs jakarta.servlet, not javax.servlet - Tomcat 9 uses the old javax namespace, so pick 10+ if you copy the imports below verbatim).

pom.xml

Only two real dependencies: the **JDBC driver** and the **Servlet API**. Gson is included just to make reading/writing JSON less painful than hand-parsing strings.

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>hourbooking-plain</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>war</packaging>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-
8</project.build.sourceEncoding>
  </properties>
```

```

<dependencies>
  <!-- Jakarta Servlet API (provided by Tomcat at runtime) -->
  <dependency>
    <groupId>jakarta.servlet</groupId>
    <artifactId>jakarta.servlet-api</artifactId>
    <version>6.0.0</version>
    <scope>provided</scope>
  </dependency>

  <!-- JDBC driver for MySQL -->
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>8.4.0</version>
  </dependency>

  <!-- JSON parsing (no framework needed) -->
  <dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.11.0</version>
  </dependency>
</dependencies>

<build>
  <finalName>hourbooking</finalName>
</build>
</project>

```

web.xml (src/main/webapp/WEB-INF/web.xml)

```

<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
  version="6.0">
  <welcome-file-list>
    <welcome-file>index.html</welcome-file>
  </welcome-file-list>
</web-app>

```

(Servlet URL mappings are declared with @WebServlet annotations directly on each servlet class below, so web.xml only needs the welcome file.)

Step 1: Database Setup (unchanged)

Same schema as the Spring version - run this once in MySQL Workbench:

```

CREATE DATABASE IF NOT EXISTS hour_booking_db;
USE hour_booking_db;

CREATE TABLE users (
  username VARCHAR(50) PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  password VARCHAR(255) NOT NULL,
  role VARCHAR(20) NOT NULL
);

CREATE TABLE staff_table (
  member_id VARCHAR(50) PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  base_rate DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (member_id) REFERENCES users(username) ON DELETE
CASCADE
);

```

```

CREATE TABLE project_table (
    wbs_code VARCHAR(50) PRIMARY KEY,
    project_name VARCHAR(100) NOT NULL,
    expiry_date DATE NOT NULL,
    amount_allocated DECIMAL(15,2) NOT NULL,
    amount_available DECIMAL(15,2) NOT NULL,
    amount_booked DECIMAL(15,2) DEFAULT 0.00
);

CREATE TABLE project_member_mapping_table (
    mapping_id INT AUTO_INCREMENT PRIMARY KEY,
    wbs_code VARCHAR(50),
    member_id VARCHAR(50),
    allocation_percentage DECIMAL(5,2),
    FOREIGN KEY (wbs_code) REFERENCES project_table(wbs_code),
    FOREIGN KEY (member_id) REFERENCES staff_table(member_id)
);

CREATE TABLE booking_table (
    booking_id INT AUTO_INCREMENT PRIMARY KEY,
    wbs_code VARCHAR(50),
    member_id VARCHAR(50),
    hours_booked DECIMAL(6,2),
    month INT,
    year INT,
    calculated_cost DECIMAL(15,2),
    status VARCHAR(30) DEFAULT 'Pending Review',
    FOREIGN KEY (wbs_code) REFERENCES project_table(wbs_code),
    FOREIGN KEY (member_id) REFERENCES staff_table(member_id)
);

INSERT INTO users VALUES ('staff01', 'John Doe', 'staff123',
'staff');
INSERT INTO staff_table VALUES ('staff01', 'John Doe', 500.00);
INSERT INTO project_table VALUES ('WBS101', '2D Active Missile
Detection Radar', '2026-12-31', 100000.00, 100000.00, 0.00);
INSERT INTO project_member_mapping_table (wbs_code, member_id,
allocation_percentage) VALUES ('WBS101', 'staff01', 25.00);

```

Step 2: Raw JDBC Connection Helper

This is the plain-Java equivalent of Spring's `spring.datasource.*` properties: one class that opens a `java.sql.Connection` using `DriverManager`.

```

package com.example.hourbooking.db;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {
    private static final String URL =
        "jdbc:mysql://localhost:3306/hour_booking_db?
useSSL=false&serverTimezone=UTC";
    private static final String USER = "root";
    private static final String PASSWORD = "your_mysql_password";

    static {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            throw new RuntimeException("MySQL JDBC driver not
found", e);
        }
    }
}

```

```

        public static Connection getConnection() throws SQLException {
            return DriverManager.getConnection(URL, USER, PASSWORD);
        }
    }

```

Every DAO below calls DBConnection.getConnection() and uses PreparedStatement directly - there is no ORM translating objects to SQL, so the queries are explicit.

Step 3: DAO Classes (raw JDBC)

dao/UserDAO.java

```

package com.example.hourbooking.dao;

import com.example.hourbooking.db.DBConnection;
import java.sql.*;

public class UserDAO {

    public boolean login(String username, String password, String[]
outRole, String[] outFullName) throws SQLException {
        String sql = "SELECT role, full_name FROM users WHERE
username = ? AND password = ?";
        try (Connection conn = DBConnection.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, username);
            ps.setString(2, password);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    outRole[0] = rs.getString("role");
                    outFullName[0] = rs.getString("full_name");
                    return true;
                }
                return false;
            }
        }
    }

    public boolean usernameExists(String username) throws
SQLException {
        String sql = "SELECT 1 FROM users WHERE username = ?";
        try (Connection conn = DBConnection.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, username);
            try (ResultSet rs = ps.executeQuery()) {
                return rs.next();
            }
        }
    }

    public void register(String username, String fullName, String
password, String role) throws SQLException {
        String sql = "INSERT INTO users (username, full_name,
password, role) VALUES (?, ?, ?, ?)";
        try (Connection conn = DBConnection.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, username);
            ps.setString(2, fullName);
            ps.setString(3, password);
            ps.setString(4, role);
            ps.executeUpdate();
        }
    }
}

```

dao/StaffDAO.java

```
package com.example.hourbooking.dao;

import com.example.hourbooking.db.DBConnection;
import java.sql.*;

public class StaffDAO {

    public void createStaff(String memberId, String name, double
baseRate) throws SQLException {
        String sql = "INSERT INTO staff_table (member_id, name,
base_rate) VALUES (?, ?, ?)";
        try (Connection conn = DBConnection.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, memberId);
            ps.setString(2, name);
            ps.setDouble(3, baseRate);
            ps.executeUpdate();
        }
    }

    public Double getBaseRate(String memberId) throws SQLException {
        String sql = "SELECT base_rate FROM staff_table WHERE
member_id = ?";
        try (Connection conn = DBConnection.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, memberId);
            try (ResultSet rs = ps.executeQuery()) {
                return rs.next() ? rs.getDouble("base_rate") : null;
            }
        }
    }
}
```

dao/ProjectDAO.java

```
package com.example.hourbooking.dao;

import com.example.hourbooking.db.DBConnection;
import java.sql.*;
import java.time.LocalDate;

public class ProjectDAO {

    public static class Project {
        public String wbsCode, projectName;
        public LocalDate expiryDate;
        public double amountAllocated, amountAvailable,
amountBooked;
    }

    public Project findByWbs(String wbsCode) throws SQLException {
        String sql = "SELECT * FROM project_table WHERE wbs_code =
?";
        try (Connection conn = DBConnection.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, wbsCode);
            try (ResultSet rs = ps.executeQuery()) {
                if (!rs.next()) return null;
                Project p = new Project();
                p.wbsCode = rs.getString("wbs_code");
                p.projectName = rs.getString("project_name");
                p.expiryDate =
rs.getDate("expiry_date").toLocalDate();
                p.amountAllocated =
rs.getDouble("amount_allocated");
                p.amountAvailable =
```

```

rs.getDouble("amount_available");
        p.amountBooked = rs.getDouble("amount_booked");
        return p;
    }
}

    public void create(String wbsCode, String name, LocalDate
expiry, double allocated) throws SQLException {
        String sql = "INSERT INTO project_table (wbs_code,
project_name, expiry_date, amount_allocated, amount_available,
amount_booked) VALUES (?, ?, ?, ?, ?, 0.00)";
        try (Connection conn = DBConnection.getConnection();
PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, wbsCode);
            ps.setString(2, name);
            ps.setDate(3, Date.valueOf(expiry));
            ps.setDouble(4, allocated);
            ps.setDouble(5, allocated);
            ps.executeUpdate();
        }
    }

    public void updateBookedAndAvailable(String wbsCode, double
newBooked, double newAvailable) throws SQLException {
        String sql = "UPDATE project_table SET amount_booked = ?,
amount_available = ? WHERE wbs_code = ?";
        try (Connection conn = DBConnection.getConnection();
PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setDouble(1, newBooked);
            ps.setDouble(2, newAvailable);
            ps.setString(3, wbsCode);
            ps.executeUpdate();
        }
    }
}

```

dao/MappingDAO.java

```

package com.example.hourbooking.dao;

import com.example.hourbooking.db.DBConnection;
import com.google.gson.annotations.SerializedName;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class MappingDAO {

    public static class Mapping {
        public String wbsCode, memberId;
        public double allocationPercentage;
    }

    public boolean isMapped(String wbsCode, String memberId) throws
SQLException {
        String sql = "SELECT 1 FROM project_member_mapping_table
WHERE wbs_code = ? AND member_id = ?";
        try (Connection conn = DBConnection.getConnection();
PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, wbsCode);
            ps.setString(2, memberId);
            try (ResultSet rs = ps.executeQuery()) {
                return rs.next();
            }
        }
    }
}

```

```

        public void map(String wbsCode, String memberId, double
allocationPercentage) throws SQLException {
            String sql = "INSERT INTO project_member_mapping_table
(wbs_code, member_id, allocation_percentage) VALUES (?, ?, ?)";
            try (Connection conn = DBConnection.getConnection();
                PreparedStatement ps = conn.prepareStatement(sql)) {
                ps.setString(1, wbsCode);
                ps.setString(2, memberId);
                ps.setDouble(3, allocationPercentage);
                ps.executeUpdate();
            }
        }

        public List<Mapping> findByMember(String memberId) throws
SQLException {
            String sql = "SELECT * FROM project_member_mapping_table
WHERE member_id = ?";
            List<Mapping> results = new ArrayList<>();
            try (Connection conn = DBConnection.getConnection();
                PreparedStatement ps = conn.prepareStatement(sql)) {
                ps.setString(1, memberId);
                try (ResultSet rs = ps.executeQuery()) {
                    while (rs.next()) {
                        Mapping m = new Mapping();
                        m.wbsCode = rs.getString("wbs_code");
                        m.memberId = rs.getString("member_id");
                        m.allocationPercentage =
rs.getDouble("allocation_percentage");
                        results.add(m);
                    }
                }
            }
            return results;
        }
    }
}

```

dao/BookingDAO.java

```

package com.example.hourbooking.dao;

import com.example.hourbooking.db.DBConnection;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class BookingDAO {

    public static class Booking {
        public int bookingId;
        public String wbsCode, memberId, status;
        public double hoursBooked, calculatedCost;
        public int month, year;
    }

    public void insert(String wbsCode, String memberId, double
hours, int month, int year, double cost) throws SQLException {
        String sql = "INSERT INTO booking_table (wbs_code,
member_id, hours_booked, month, year, calculated_cost, status)
VALUES (?, ?, ?, ?, ?, ?, 'Pending Review')";
        try (Connection conn = DBConnection.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, wbsCode);
            ps.setString(2, memberId);
            ps.setDouble(3, hours);
            ps.setInt(4, month);
            ps.setInt(5, year);
            ps.setDouble(6, cost);
            ps.executeUpdate();
        }
    }
}

```

```

    }
}

public List<Booking> findAll() throws SQLException {
    String sql = "SELECT * FROM booking_table";
    List<Booking> results = new ArrayList<>();
    try (Connection conn = DBConnection.getConnection();
        Statement st = conn.createStatement();
        ResultSet rs = st.executeQuery(sql)) {
        while (rs.next()) {
            Booking b = new Booking();
            b.bookingId = rs.getInt("booking_id");
            b.wbsCode = rs.getString("wbs_code");
            b.memberId = rs.getString("member_id");
            b.hoursBooked = rs.getDouble("hours_booked");
            b.month = rs.getInt("month");
            b.year = rs.getInt("year");
            b.calculatedCost = rs.getDouble("calculated_cost");
            b.status = rs.getString("status");
            results.add(b);
        }
    }
    return results;
}
}

```

Step 4: Servlets (replace the Spring @RestController)

Each servlet reads the JSON body with Gson, calls the DAO directly (raw JDBC, no repository/Hibernate layer), and writes a JSON response. @WebServlet sets the URL so the frontend's fetch() calls barely change.

servlet/LoginServlet.java

```

package com.example.hourbooking.servlet;

import com.example.hourbooking.dao.UserDAO;
import com.google.gson.Gson;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;
import java.io.IOException;
import java.util.Map;

@WebServlet("/api/login")
public class LoginServlet extends HttpServlet {
    private final UserDAO userDAO = new UserDAO();
    private final Gson gson = new Gson();

    @Override
    protected void doPost(HttpServletRequest req,
        HttpServletResponse resp) throws IOException {
        Map<?, ?> body = gson.fromJson(req.getReader(), Map.class);
        String username = (String) body.get("username");
        String password = (String) body.get("password");

        String[] role = new String[1];
        String[] fullName = new String[1];
        resp.setContentType("application/json");
        try {
            boolean ok = userDAO.login(username, password, role,
                fullName);
            if (ok) {
                resp.getWriter().write(gson.toJson(Map.of(

```

```

        "username", username, "role", role[0],
        "fullName", fullName[0]));
    } else {
        resp.setStatus(401);
        resp.getWriter().write("Invalid credentials");
    }
} catch (Exception e) {
    resp.setStatus(500);
    resp.getWriter().write("Server error: " +
e.getMessage());
}
}
}

```

servlet/RegisterServlet.java

```

package com.example.hourbooking.servlet;

import com.example.hourbooking.dao.StaffDAO;
import com.example.hourbooking.dao.UserDAO;
import com.google.gson.Gson;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;
import java.io.IOException;
import java.util.Map;

@WebServlet("/api/register")
public class RegisterServlet extends HttpServlet {
    private final UserDAO userDAO = new UserDAO();
    private final StaffDAO staffDAO = new StaffDAO();
    private final Gson gson = new Gson();

    @Override
    protected void doPost(HttpServletRequest req,
HttpServletResponse resp) throws IOException {
        Map<?, ?> body = gson.fromJson(req.getReader(), Map.class);
        String username = (String) body.get("username");
        String fullName = (String) body.get("fullName");
        String password = (String) body.get("password");
        String role = (String) body.get("role");

        try {
            if (userDAO.usernameExists(username)) {
                resp.setStatus(400);
                resp.getWriter().write("Username already exists!");
                return;
            }
            userDAO.register(username, fullName, password, role);
            if ("staff".equalsIgnoreCase(role)) {
                staffDAO.createStaff(username, fullName, 500.00);
            }
            resp.getWriter().write("Registration successful");
        } catch (Exception e) {
            resp.setStatus(500);
            resp.getWriter().write("Server error: " +
e.getMessage());
        }
    }
}

```

servlet/BookHoursServlet.java

This carries over the same business rules as the Spring version - expiry lock, staff mapping validation, cost calculation - just written against raw JDBC instead of JPA.

```

package com.example.hourbooking.servlet;

```

```

import com.example.hourbooking.dao.*;
import com.google.gson.Gson;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;
import java.io.IOException;
import java.time.LocalDate;
import java.util.Map;

@WebServlet("/api/book-hours")
public class BookHoursServlet extends HttpServlet {
    private final ProjectDAO projectDAO = new ProjectDAO();
    private final MappingDAO mappingDAO = new MappingDAO();
    private final StaffDAO staffDAO = new StaffDAO();
    private final BookingDAO bookingDAO = new BookingDAO();
    private final Gson gson = new Gson();

    @Override
    protected void doPost(HttpServletRequest req,
        HttpServletResponse resp) throws IOException {
        Map<?, ?> body = gson.fromJson(req.getReader(), Map.class);
        String wbsCode = (String) body.get("wbsCode");
        String memberId = (String) body.get("memberId");
        double hours = ((Number)
body.get("hoursBooked")).doubleValue();
        int month = ((Number) body.get("month")).intValue();
        int year = ((Number) body.get("year")).intValue();

        try {
            ProjectDAO.Project project =
projectDAO.findByWbs(wbsCode);
            if (project == null) {
                resp.setStatus(400);
                resp.getWriter().write("Invalid WBS Code");
                return;
            }

            // Expiry Lock Check
            if (LocalDate.now().isAfter(project.expiryDate)) {
                resp.setStatus(400);
                resp.getWriter().write("Project is expired! Booking
blocked.");
                return;
            }

            // Staff Mapping Validation
            if (!mappingDAO.isMapped(wbsCode, memberId)) {
                resp.setStatus(400);
                resp.getWriter().write("Staff is not mapped to this
project.");
                return;
            }

            Double baseRate = staffDAO.getBaseRate(memberId);
            double cost = hours * baseRate; // Financial Calculation

            bookingDAO.insert(wbsCode, memberId, hours, month, year,
cost);

            double newBooked = project.amountBooked + cost;
            double newAvailable = project.amountAllocated -
newBooked;
            projectDAO.updateBookedAndAvailable(wbsCode, newBooked,
newAvailable);

            resp.getWriter().write("Hours booked successfully and
marked as Pending Review!");
        } catch (Exception e) {
            resp.setStatus(500);
            resp.getWriter().write("Server error: " +

```

```

e.getMessage());
    }
}

```

servlet/ProjectServlet.java

```

package com.example.hourbooking.servlet;

import com.example.hourbooking.dao.ProjectDAO;
import com.google.gson.Gson;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;
import java.io.IOException;
import java.time.LocalDate;
import java.util.Map;

@WebServlet("/api/projects")
public class ProjectServlet extends HttpServlet {
    private final ProjectDAO projectDAO = new ProjectDAO();
    private final Gson gson = new Gson();

    @Override
    protected void doPost(HttpServletRequest req,
        HttpServletResponse resp) throws IOException {
        Map<?, ?> body = gson.fromJson(req.getReader(), Map.class);
        String wbsCode = (String) body.get("wbsCode");
        String name = (String) body.get("projectName");
        LocalDate expiry = LocalDate.parse((String)
body.get("expiryDate"));
        double allocated = ((Number)
body.get("amountAllocated")).doubleValue();

        try {
            projectDAO.create(wbsCode, name, expiry, allocated);
            resp.getWriter().write("Project created successfully");
        } catch (Exception e) {
            resp.setStatus(500);
            resp.getWriter().write("Server error: " +
e.getMessage());
        }
    }
}

```

servlet/MapMemberServlet.java

```

package com.example.hourbooking.servlet;

import com.example.hourbooking.dao.MappingDAO;
import com.google.gson.Gson;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;
import java.io.IOException;
import java.util.Map;

@WebServlet("/api/map-member")
public class MapMemberServlet extends HttpServlet {
    private final MappingDAO mappingDAO = new MappingDAO();
    private final Gson gson = new Gson();

    @Override
    protected void doPost(HttpServletRequest req,
        HttpServletResponse resp) throws IOException {
        Map<?, ?> body = gson.fromJson(req.getReader(), Map.class);
        String wbsCode = (String) body.get("wbsCode");
        String memberId = (String) body.get("memberId");
        double allocation = ((Number)

```

```

body.get("allocationPercentage")).doubleValue();

        try {
            mappingDAO.map(wbsCode, memberId, allocation);
            resp.getWriter().write("Member mapped successfully");
        } catch (Exception e) {
            resp.setStatus(500);
            resp.getWriter().write("Server error: " +
e.getMessage());
        }
    }
}

```

servlet/BookingsServlet.java

```

package com.example.hourbooking.servlet;

import com.example.hourbooking.dao.BookingDAO;
import com.google.gson.Gson;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;
import java.io.IOException;

@WebServlet("/api/bookings")
public class BookingsServlet extends HttpServlet {
    private final BookingDAO bookingDAO = new BookingDAO();
    private final Gson gson = new Gson();

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse
resp) throws IOException {
        resp.setContentType("application/json");
        try {
            resp.getWriter().write(gson.toJson(bookingDAO.findAll()));
        } catch (Exception e) {
            resp.setStatus(500);
            resp.getWriter().write("Server error: " +
e.getMessage());
        }
    }
}

```

Step 5: Frontend (unchanged)

Copy index.html, register.html, staff-dashboard.html, admin-dashboard.html, style.css, and script.js from the Spring Boot version as-is into src/main/webapp/. Since every servlet above keeps the same /api/... URL pattern (/api/login, /api/register, /api/book-hours, /api/projects, /api/map-member, /api/bookings), the existing script.js fetch() calls work without modification - only change the base URL from http://localhost:8080 to wherever Tomcat serves the app (commonly http://localhost:8080/hourbooking when deployed as a WAR named hourbooking).

Step 6: How to Run

1. **Database:** run the Step 1 SQL script in MySQL Workbench (same as before).
2. **Build:** mvn clean package - produces target/hourbooking.war.
3. **Deploy:** drop hourbooking.war into Tomcat's webapps/ folder (or deploy through your IDE's Tomcat integration) and start Tomcat.
4. **Browse:** go to http://localhost:8080/hourbooking/index.html.

What changed vs. the Spring Boot version

Spring Boot version	Plain Java version
@RestController	@WebServlet + HttpServlet
Spring Data JPA JpaRepository	Hand-written DAO classes
Hibernate generates SQL	You write every SQL statement yourself
spring.datasource.* properties	DriverManager.getConnection(...) in DBConnection
Embedded Tomcat (runs itself)	You deploy the WAR to your own Tomcat
mvn spring-boot:run	mvn clean package then deploy the WAR

JDBC is used explicitly here - every DAO method opens a Connection, builds a PreparedStatement, and runs it directly against hour_booking_db. There is no ORM layer translating between objects and SQL.